09｜开发环境搭建：大模型应用项目初始化

无

通过这一讲的学习，你将获得一个可复用、可扩展、生产就绪的大模型应用项目模板，为后续实现更多功能打下坚实基础。

0:00 / 7:30

你好，我是袁从德。

前八讲，我们完成了从宏观认知到核心技术的系统性构建。从大模型应用的整体开发路径，到情感聊天场景的价值选择；从Transformer底层原理，到Prompt工程、上下文管理、Agent架构与RAG技术的层层递进——我们已经绘制出了一幅完整的“大模型应用开发全景图”。

然而，所有的理论认知与方法论，最终都必须落地于一行行可运行的代码、一个个可交互的功能模块。**知道做什么是智慧，知道怎么做是能力，而真正做到才是价值的起点。**

因此，从今天开始，我们正式进入课程的**实战篇（一）：情感聊天应用基础搭建**。这不仅是技术旅程的转折点，更是思维方式的跃迁——从理解AI走向创造AI。通过这一讲的学习，你将获得一个**可复用、可扩展、生产就绪的大模型应用项目模板**，为后续实现记忆管理、安全过滤、Agent行为编排、RAG知识接入等功能打下坚实基础。

更重要的是，你将建立起“工程化思维”——不再把大模型当作一个孤立的API调用，而是将其视为一个复杂智能系统的有机组成部分。这种思维，正是从“调用者”迈向“系统架构师”的分水岭。

现在，让我们正式启程，从零开始，搭建属于你的第一个大模型应用项目。

今天我们的核心任务是：**完成开发环境的搭建与项目初始化**。看似平凡，实则至关重要。一个清晰、规范、可扩展的项目结构，就像一座高楼的地基，决定了后续开发的效率、协作的顺畅与系统的稳定性。

我们将以“心语”心理健康陪伴机器人为具体项目案例，手把手带你完成以下关键步骤：

- 本地开发环境的准备与配置
- 项目目录结构的设计与初始化
- 核心依赖库的选型与安装
- 大模型API的接入与测试
- 配置管理与安全实践
- 第一个可运行的“Hello, AI”对话原型

一、开发环境准备：打造高效稳定的AI开发工作台

在开始编码之前，我们需要先为项目准备好一个稳定、隔离且高效的开发环境。这不仅能避免依赖冲突，还能提升团队协作效率，是专业开发的第一课。

1.1 技术栈选型：为什么是Python + FastAPI?

在大模型应用开发中，Python凭借其丰富的AI生态（如LangChain、LlamaIndex、Hugging Face等）和简洁的语法，已成为事实上的标准语言。我们选择 **Python 3.10+** 作为主语言，确保兼容最新库版本。

后端框架我们选用 **FastAPI**，原因如下：

- **高性能**：基于ASGI（异步服务器网关接口），支持异步请求处理，适合大模型的高延迟IO场景；

- **自动文档生成**：集成Swagger UI，便于接口调试与团队协作；
- **类型提示友好**：与Pydantic深度集成，提升代码可读性与健壮性；
- **轻量灵活**：无过多约定，适合快速迭代的AI应用开发。

  替代方案：如果你更熟悉Flask或Django，也可使用，但建议优先尝试FastAPI以体验异步优势。

1.2 环境管理：使用venv或conda创建虚拟环境

为了避免不同项目间的依赖冲突，我们必须使用虚拟环境。

使用venv（推荐新手）：

复制代码

```
# 创建虚拟环境
python -m venv venv

# 激活虚拟环境（Linux/Mac）
source venv/bin/activate

# 激活虚拟环境（Windows）
venv\Scripts\activate
```

使用conda（适合数据科学背景）：

复制代码

```
# 创建环境
conda create -n emotion-chat python=3.10

# 激活环境
conda activate emotion-chat
```

激活后，终端前缀应显示环境名，如venv或 emotion-chat，表示环境已生效。

1.3 包管理：pip与requirements.txt

我们使用pip安装依赖，并通过requirements.txt锁定版本，确保环境一致性。

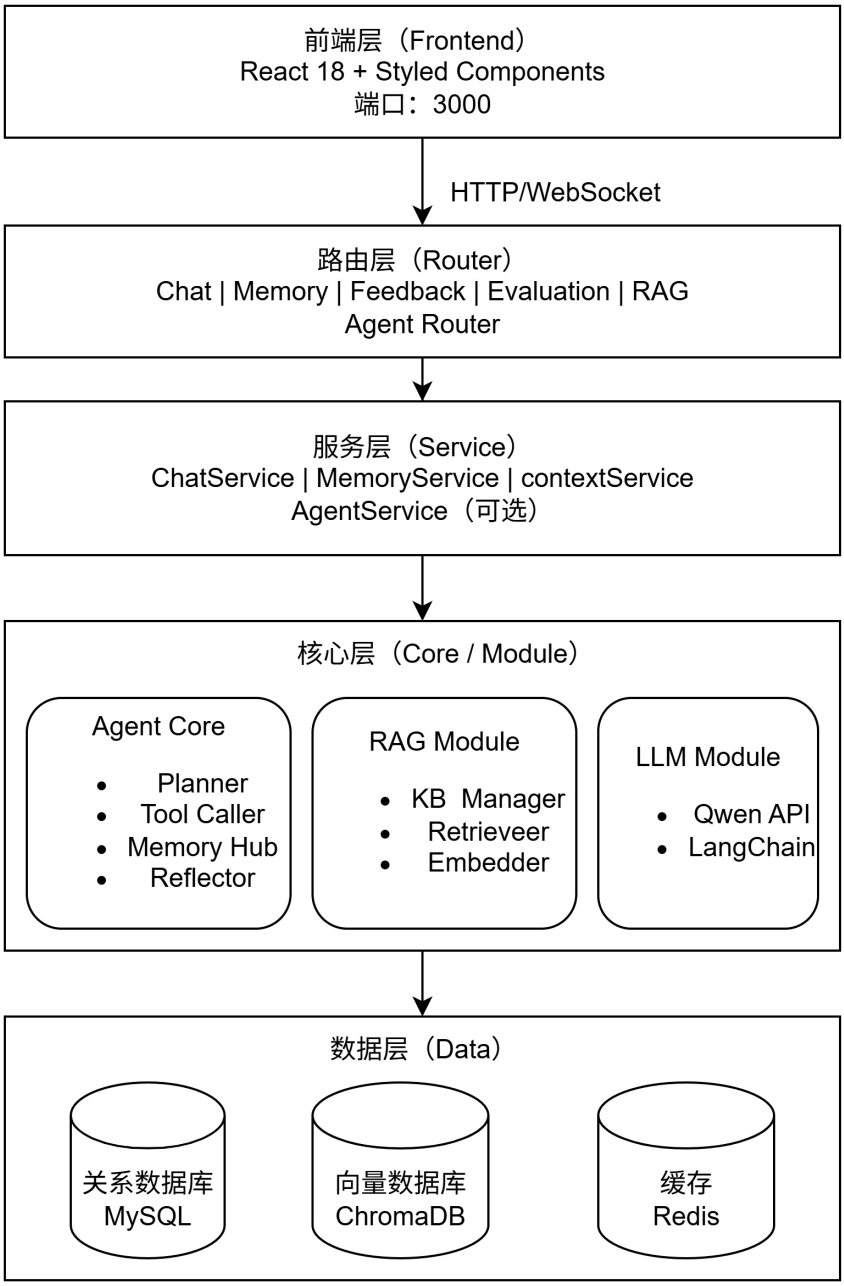
初始化项目后，创建空文件：

```
touch requirements.txt
```

复制代码

二、项目初始化：构建清晰可扩展的目录结构

良好的项目架构是可维护性的基石。我们为“心语”机器人设计，整体架构如图9-1所示。



极客时间

说明：此结构遵循“分层架构”思想，分离关注点，便于后期扩展Agent、RAG、向量数据库等模块。

代码：https://github.com/congde/emotional_chat

接下来，我们详细介绍分层服务架构。

2.1 路由层 (Router Layer)

路由层负责API接口定义和请求分发。

复制代码

```
backend/routers/
├── chat.py           - 聊天接口
├── memory.py         - 记忆管理接口
├── feedback.py       - 用户反馈接口
├── evaluation.py     - 评估系统接口
└── agent_router.py   - Agent智能接口
```

2.2 服务层 (Service Layer)

服务层用于业务逻辑封装和编排。

复制代码

```
backend/services/
├── chat_service.py       - 聊天服务
├── memory_service.py     - 记忆服务
├── context_service.py    - 上下文服务
└── agent_service.py      - Agent服务（可选）
```

2.3 核心层 (Core/Module Layer)

核心功能模块如下所示。

复制代码

```
backend/modules/
├── agent/           - Agent智能核心
│   ├── core/        - Agent Core, Planner, Tool Caller等
│   ├── services/    - Agent服务实现
│   └── routers/     - Agent路由
├── rag/             - RAG知识库系统
```

```
│   ├── core/        - 知识库管理、检索
│   ├── services/    - RAG服务
│   └── routers/     - RAG路由
└── llm/             - LLM模型封装
    └── qwen_client.py - 通义千问客户端
```

2.4 数据层 (Data Layer)

数据层用于数据存储和管理，涉及到的技术框架包括后面这些。

- MySQL: 用户数据、对话历史、情感记录
- ChromaDB: 向量存储、语义检索
- Redis: 缓存、会话状态（生产环境）

我们来看看MySQL数据库表结构。

1. users - 用户表

复制代码

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id VARCHAR(100) UNIQUE NOT NULL,
    username VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CU
    is_active BOOLEAN DEFAULT TRUE
);
```

2. chat_sessions - 会话表

```
CREATE TABLE chat_sessions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    session_id VARCHAR(100) UNIQUE NOT NULL,
    user_id VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CU
    is_active BOOLEAN DEFAULT TRUE
);
```

3. chat_messages - 消息表

```
CREATE TABLE chat_messages (
    id INT PRIMARY KEY AUTO_INCREMENT,
    session_id VARCHAR(100) NOT NULL,
    user_id VARCHAR(100) NOT NULL,
    role VARCHAR(20) NOT NULL,  -- user, assistant
    content TEXT NOT NULL,
    emotion VARCHAR(50),
    emotion_intensity FLOAT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. emotion_analysis - 情感分析表

```
CREATE TABLE emotion_analysis (
    id INT PRIMARY KEY AUTO_INCREMENT,
    session_id VARCHAR(100) NOT NULL,
    user_id VARCHAR(100) NOT NULL,
    message_id INT NOT NULL,
```

```
    emotion VARCHAR(50) NOT NULL,
    intensity FLOAT NOT NULL,
    keywords TEXT,
    suggestions TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5. knowledge - 知识库表

复制代码

```
CREATE TABLE knowledge (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    content TEXT NOT NULL,
    category VARCHAR(100),
    tags TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CU
    is_active BOOLEAN DEFAULT TRUE
);
```

6. system_logs - 系统日志表

复制代码

```
CREATE TABLE system_logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    level VARCHAR(20) NOT NULL,
    message TEXT NOT NULL,
    session_id VARCHAR(100),
    user_id VARCHAR(100),
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.5 技术栈

接下来是技术栈的选择，你可以参考后面的表格9-1。

	技术栈
后端技术	• Python 3.8+：主要开发语言 • FastAPI 0.83+：现代化的 Web 框架 • LangChain 0.1+：对话流管理和记忆管理 • Pydantic：数据验证和序列化
AI/大模型	• 阿里云通义千问 (Qwen Plus)：大语言模型 • LangChain-OpenAI：LLM 集成 • Sentence Transformers：文本嵌入模型
数据存储	• MySQL 8.0：关系型数据库（用户数据、对话历史） • ChromaDB 0.4+：向量数据库（语义检索、长期记忆） • Redis 7：缓存和会话管理（生产环境） • SQLAlchemy 1.4+：ORM 框架 • Alembic：数据库迁移工具
文档处理	• PyPDF 3.17+：PDF 文档解析 • python-multipart：文件上传支持
运维工具	• Docker & Docker Compose：容器化部署 • Prometheus：指标监控 • Grafana：数据可视化 • Elasticsearch & Kibana：日志聚合和分析 • Nginx：反向代理
前端技术	• React 18：UI 框架 • Styled Components：CSS-in-JS 样式方案 • Framer Motion：动画效果库 • Axios：HTTP客户端

极客时间

三、配置管理：安全、灵活的项目“控制台”

硬编码API密钥是重大安全隐患。我们使用配置文件 + 环境变量的方式管理敏感信息。

3.1 创建config.env文件（本地）

先来创建config.env文件，代码如下所示。

复制代码

```
# config.env
# API配置 - 使用阿里云通义千问(Qwen)
DASHSCOPE_API_KEY={填写API Key}
API_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1

# LangChain配置
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_langchain_key_here

# 数据库配置
DATABASE_URL=mysql+pymysql://root:emotional_chat_2025@localhost

# MySQL数据库配置
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=emotional_chat_2025
MYSQL_DATABASE=emotional_chat

# 向量数据库配置
CHROMA_PERSIST_DIRECTORY=./chroma_db

# 模型配置 - 使用通义千问
DEFAULT_MODEL=qwen-plus
```

```
TEMPERATURE=0.7
MAX_TOKENS=1000

# 服务器配置
HOST=0.0.0.0
PORT=8000
DEBUG=true
```

⚠️ 注意：config.env文件不应提交到Git，需在 .gitignore 中添加：

复制代码

```
# .gitignore
# Node modules
frontend/node_modules/
node_modules/

# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
ENV/
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
```

```
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# IDE
.vscode/
.idea/
*.swp
*.swo
*~

# Environment variables
.env
*.env.local
config.env

# Logs
*.log
log/
!log/.gitkeep

# Database
*.db
```

```
*.sqlite3
chroma_db/
```

3.2 配置加载：config.py

之后来完成配置加载。

复制代码

```
import os
from pathlib import Path
from dotenv import load_dotenv

# 加载.env配置文件
env_path = Path(__file__).parent / 'config.env'
load_dotenv(env_path)

# 获取项目根目录
PROJECT_ROOT = os.getenv('PROJECT_ROOT', str(Path(__file__).p

class Config:
    # OpenAI配置
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

    # LangChain配置
    LANGCHAIN_TRACING_V2 = os.getenv("LANGCHAIN_TRACING_V2",
    LANGCHAIN_API_KEY = os.getenv("LANGCHAIN_API_KEY")

    # 数据库配置
    DATABASE_URL = os.getenv("DATABASE_URL")
    MYSQL_HOST = os.getenv("MYSQL_HOST", "localhost")
    MYSQL_PORT = int(os.getenv("MYSQL_PORT", "3306"))
```

```
    MYSQL_USER = os.getenv("MYSQL_USER", "root")
    MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD")
    MYSQL_DATABASE = os.getenv("MYSQL_DATABASE", "emotional_c

    # 向量数据库配置（使用项目根目录的绝对路径）
    CHROMA_PERSIST_DIRECTORY = os.getenv("CHROMA_PERSIST_DIRE
                                        os.path.join(PROJECT_

    # 模型配置
    DEFAULT_MODEL = os.getenv("DEFAULT_MODEL", "gpt-4")
    TEMPERATURE = float(os.getenv("TEMPERATURE", "0.7"))
    MAX_TOKENS = int(os.getenv("MAX_TOKENS", "1000"))

    # 服务器配置
    HOST = os.getenv("HOST", "0.0.0.0")
    PORT = int(os.getenv("PORT", "8000"))
    DEBUG = os.getenv("DEBUG", "false").lower() == "true"
```

四、启动与测试：见证AI的第一次回应

下面我们完成启动和测试工作。

复制代码

```
# 获取最新代码
git clone https://github.com/congde/emotional_chat.git

# 完成相关依赖安装部署任务，后启动前后端
sh +x restart_services.sh
```

打开浏览器访问后面的地址：http://localhost:3000/

你将看到自动生成的API文档界面，如图9-2所示。

如果出现了这个界面，那么恭喜你！你已成功构建并运行了第一个大模型应用！

五、结语：从“0”到“1”，开启智能应用的创造之旅

当我们成功运行第一个AI对话时，我们完成的不仅是技术上的“破冰”，更是一次认知上的跃迁。

你亲手搭建的，不再只是一个空项目模板，而是一个**具备完整MVP能力的情感聊天机器人雏形**。它已经能够：

- 安全地管理配置与密钥
- 通过标准化API接收用户输入
- 调用大模型生成自然语言回复
- 返回结构化结果与使用统计

更重要的是，你建立了一套**工程化开发范式**：

- 分层架构 → 提升可维护性
- 配置分离 → 保障安全性
- 异步设计 → 支持高并发
- 日志监控 → 便于调试优化

这正是大模型应用开发的“基础设施”。有了它，我们后面才能从容地为AI注入记忆（向量数据库），赋予人格（Prompt工程），建立安全护栏（内容过滤），最终走向自主行动（Agent）。

记住，每一个伟大的AI产品，都始于这样一个简单的main.py文件。你今天写下的每一行代码，都是在为未来那个能真正理解人类、陪伴人类的“数字生命”添砖加瓦。

现在，我们的地基已经打好。接下来，让我们为“心语”注入记忆与情感，让它从对话机器真正成长为陪伴伙伴。

思考题

> “一个好的项目结构不仅仅是代码的组织方式，更是对系统演进路径的设计。”
> 在这一讲中，我们为“心语”心理健康陪伴机器人搭建了一个结构清晰、可扩展的项目框架，并强调了工程化思维在大模型应用开发中的重要性。请结合课程内容，回答以下问题。
>
> 请从分层架构设计、配置安全管理和技术栈选型三个角度，分析本这一讲所构建的项目模板如何为后续实现“记忆管理”（RAG）、“Agent行为编排”和“生产环境部署”等高级功能提供支持与扩展性保障。

提示思考方向：

- 分层架构中，RAG和Agent的独立模块设计带来了哪些解耦优势？
- 配置分离与环境变量管理如何提升应用在不同环境（开发/测试/生产）下的安全性和灵活性？

希望通过今天的学习，让你对情感机器人的开发全程有个大致认识。如果有任何疑问，期待你在留言区和我交流。

评论区

评论 #1

Dream. · 2025-12-01 00:33:05 · 点赞 4

> github上的仓库看起来都已经实现的差不多了。这个实战篇的最终目标就是构建成现在的仓库的样子吗？如果是这样的话，我就放心的去学习githu仓库了

讨论串（2）

1. **讨论 #1**

   袁从德 · 2025-12-02 09:58:25 · 点赞 1

   是的，实战篇的最终目标就是构建成现在的仓库的样子
   仓库会持续更新，建议多看docs下的文档，都是最新一手资料

2. **讨论 #2**

   tangdong · 2025-12-01 11:21:39 · 点赞 0

   你们用的啥IDE，vscode？

评论 #2

tangdong · 2025-12-01 16:58:57 · 点赞 2

> 有没有windows环境下的环境搭建和开发指南

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-02 09:56:46 · 点赞 0

   有
   https://github.com/congde/emotional_chat/blob/main/docs/Windows%E8%BF

%90%E8%A1%8C%E6%8C%87%E5%8D%97.md

评论 #3

学习使我快乐 · 2025-12-08 13:49:40 · 点赞 1

> 老师，用了最新的requirements安装后，报错了，Ubuntu20.04系统，ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
> grpcio-status 1.51.1 requires protobuf>=4.21.6, but you have protobuf 3.20.3 which is incompatible.
> langchain-huggingface 0.1.2 requires langchain-core<0.4.0,>=0.3.15, but you have langchain-core 0.2.43 which is incompatible.
> pydantic-settings 2.6.1 requires pydantic>=2.7.0, but you have pydantic 1.10.24 which is incompatible.

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-08 16:48:34 · 点赞 0

   谢谢反馈，强烈建议使用虚拟环境来避免依赖冲突和污染系统 Python 环境。

   https://github.com/congde/emotional_chat/blob/main/PYTHON310_INSTALL.md

评论 #4

墨名次 · 2025-12-02 22:44:13 · 点赞 1

> 老师是不是忘记上传requirements-py310.txt了，

讨论串（1）

1. **讨论 #1**

袁从德 · 2025-12-03 17:36:51 · 点赞 1

这个工程，还在加新的组件，要同时维护python3.8和python3.10很麻烦，考虑mac用不了python3.8，现在统一用python3.10 requirements.txt

评论 #5

Geek_915441 · 2025-12-02 17:50:47 · 点赞 1

> Rag和agent模块独立模块设计，方便独立维护，升级或者替换，还可以多场景复用。
> 配置分离，开发阶段专注于调试，不用频繁调整环境。上线时，可以轻松切换，既可以控制误操作带来的风险，也避免了敏感信息的泄漏。

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-03 18:26:03 · 点赞 0

   你的理解非常到位！👍

   RAG 和 Agent 的独立模块设计，确实实现了高内聚、低耦合，不仅便于单独测试、迭代和替换，也为未来在其他项目中复用打下基础。

   配置分离机制则有效保障了安全性与部署灵活性，让开发、测试、生产环境切换更可靠，也避免了密钥等敏感信息泄露的风险。

   这正是工程化思维的核心体现：为变化而设计，为协作而规范，为安全而隔离。

评论 #6

德德 · 2025-12-02 08:50:21 · 点赞 1

> 项目创建和依赖管理是不是可以用 uv

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-02 09:46:18 · 点赞 0

   可以

   我今天更新一下代码

评论 #7

我知道一条近路 · 2025-12-01 18:02:08 · 点赞 1

> 老师您好，我发现 GitHub 的 repo 里有一些重复代码，有没有考虑过可以通过 branch 或者 tag 来区分一下版本呢?

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-02 09:57:15 · 点赞 0

   谢谢反馈，我后面区分一下

评论 #8

Kingkong Wang · 2025-12-21 20:47:46 · 点赞 0

> windows下使用alembic upgrade head报错，好像是定义得metadata 字段，是内部保留的名字

讨论串（1）

1. **讨论 #1**

   袁从德 · 2025-12-22 10:22:22 · 点赞 0

   谢谢反馈，已修复

   [root@ emotional_chat]# alembic current

   INFO [alembic.runtime.migration] Context impl MySQLImpl.

   INFO [alembic.runtime.migration] Will assume non-transactional DDL.

   002

[root@ emotional_chat]# alembic history

002 -> 003 (head), rename metadata to extra_metadata

001 -> 002, add response evaluation table

<base> -> 001, Initial schema

[root@ emotional_chat]# alembic upgrade head

INFO [alembic.runtime.migration] Context impl MySQLImpl.

INFO [alembic.runtime.migration] Will assume non-transactional DDL.

INFO [alembic.runtime.migration] Running upgrade 002 -> 003, rename metadata to extra_metadata

评论 #9

June · 2026-02-23 01:41:57 · 点赞 1

> 今天花了几个小时搭建开发环境，总结了自己踩过的三个大坑，也算是给自己的一个记录：
> 1、盲目追求新版本：为了省事直接去官网下载了最新的Python，结果导致了兼容性问题，反而浪费了更多时间去排查。
> 2、忽视下载源配置：在安装依赖时没有提前配置好国内镜像源，导致下载速度极慢，浪费了时间。
> 3、缺乏虚拟环境意识：完全不懂虚拟环境的概念，导致全局环境混乱，最后只能通过反复删除重装来解决，走了很多弯路。

评论 #10

pp · 2026-03-09 18:10:39 · 点赞 0

> 这个介绍跟后面这个有点乱； 是一步一步实现仓库那样的 还是直接拉代码下来的

内容来自于网络，请注意甄别，个人整理学习资料，非官方课程，仅供自用，切勿传播，违反概不负责